

2.ÖDEV-> Prokudin-Gorskii Renkli Fotoğraf Restorasyonu

BEYZA GÜLER

230212044

YAPAY ZEKA MÜHENDİSLİĞİ

Prokudin-Gorskii Renkli Fotoğraf Restorasyonu

1. Ortamın Hazırlanması

Bu adımda projede kullanılacak temel kütüphaneler yüklenmiştir. google.colab modülü, Colab ortamında dosya yükleme işlemlerini gerçekleştirmek için; skimage, matplotlib ve numpy kütüphaneleri ise görüntü işleme, görselleştirme ve sayısal hesaplamalar için kullanılmıştır. Bu kütüphanelerin yüklenmesiyle, Prokudin-Gorskii renkli fotoğraf restorasyonu projesi için gerekli çalışma ortamı hazırlanmıştır.

```
from google.colab import files
from skimage import io, img_as_float, exposure
import matplotlib.pyplot as plt
import numpy as np
```

2. Görüntünün Yüklenmesi ve Kanallara Ayrılması

Bu adımda kullanıcıdan Prokudin-Gorskii'ye ait üç kanallı (B, G, R sıralı) siyah-beyaz görüntü yüklemesi istenmiştir. Yüklenen görüntü skimage kütüphanesi yardımıyla okunmuş ve 0-1 aralığına dönüştürülmüştür. Görüntü yüksekliği üç eşit parçaya bölünerek sırasıyla Mavi (Blue), Yeşil (Green) ve Kırmızı (Red) kanallar elde edilmiştir. Son olarak, bu kanallar ayrı ayrı görselleştirilerek projenin ilerleyen adımlarında hizalama ve renk birleştirme işlemlerine temel oluşturulmuştur.

```
print("Lütfen 3 kanallı (B, G, R sıralı)")
uploaded = files.upload()
filename = list(uploaded.keys())[0]

# Görüntüyü yükle
image = img_as_float(io.imread(filename))
print("", image.shape)

# Üç eşit parçaya böl (B, G, R)
height = image.shape[0] // 3
B = image[0:height, :]
G = image[height:2*height, :]
R = image[2*height:3*height, :]

# Kanalları göster
fig, ax = plt.subplots(1, 3, figsize=(12,8))
ax[0].imshow(B, cmap='gray'); ax[0].set_title("Blue Kanalı")
ax[1].imshow(G, cmap='gray'); ax[1].set_title("Green Kanalı")
ax[2].imshow(R, cmap='gray'); ax[2].set_title("Red Kanalı")
for a in ax: a.axis("off")
plt.show()
```

Lütfen 3 kanallı (B, G, R sıralı)

Dosyaları Seç Dosya seçilmedi

Saving 01880v.jpg to 01880v.jpg
(1024, 396)

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Blue Kanalı



Green Kanalı



Red Kanalı



3. Kanal Hizalama Fonksiyonlarının Tanımlanması

Bu bölümde, görüntü kanallarının birbiriyle hizalanabilmesi için gerekli fonksiyonlar tanımlanmıştır. Öncelikle SSD (Sum of Squared Differences) ve NCC (Normalized Cross-Correlation) adlı iki farklı benzerlik ölçütü oluşturulmuştur. Bu metrikler, kanallar arasındaki uyumu değerlendirerek en uygun kaydırma değerlerini bulmak için kullanılmıştır.

Ardından tanımlanan align_channel() fonksiyonu, bir kanalı referans kanal (genellikle mavi kanal) ile hizalamak amacıyla küçük kaydırmalar (-15 ile +15 piksel arası) dener. Her kaydırmada hesaplanan benzerlik skoruna göre en iyi eşleşme seçilir ve kanal, tespit edilen en uygun kaydırma miktarına göre yeniden düzenlenir. Bu adım, kanalların doğru üst üste gelmesini sağlayarak renkli görüntünün oluşturulmasına zemin hazırlar.

```
def ssd(img1, img2):
    """Sum of Squared Differences (SSD)"""
    return np.sum((img1 - img2)**2)

def ncc(img1, img2):
    """Normalized Cross-Correlation (NCC)"""
    img1_mean = img1 - np.mean(img1)
    img2_mean = img2 - np.mean(img2)
    return np.sum(img1_mean * img2_mean) / np.sqrt(np.sum(img1_mean**2) * np.sum(img2_mean**2))

def align_channel(base, channel, max_shift=15, metric='ssd'):
    """Bir kanalı referans kanala hizalar"""
    h, w = base.shape
    crop = 0.1 # Kenarları %40 kırp
    h_crop = int(h*crop)
    w_crop = int(w*crop)
    base_cropped = base[h_crop:h-h_crop, w_crop:w-w_crop]
    channel_cropped = channel[h_crop:h-h_crop, w_crop:w-w_crop]

    best_score = None
    best_shift = (0,0)

    for dx in range(-max_shift, max_shift+1):
        for dy in range(-max_shift, max_shift+1):
            shifted = np.roll(channel_cropped, shift=(dx, dy), axis=(0,1))
            if metric == 'ssd':
                score = ssd(base_cropped, shifted)
                if best_score is None or score < best_score:
                    best_score = score
                    best_shift = (dx, dy)
            elif metric == 'ncc':
                score = ncc(base_cropped, shifted)
                if best_score is None or score > best_score:
                    best_score = score
                    best_shift = (dx, dy)

    # En iyi kaydırmayı uygula
    aligned_channel = np.roll(channel, shift=best_shift, axis=(0,1))
    return aligned_channel, best_shift
```

4. Kanalların Hizalanması ve Renkli Görüntünün Oluşturulması

Bu adımda, daha önce tanımlanan hizalama fonksiyonları kullanılarak yeşil (G) ve kırmızı (R) kanallar, referans olarak seçilen mavi (B) kanala göre hizalanmıştır. Hizalama işlemi sırasında her kanal için en uygun kaydırma miktarı (shift_G ve shift_R) belirlenmiş ve ekrana yazdırılmıştır.

Ardından hizalanmış kanallar bir araya getirilerek RGB formatında renkli görüntü oluşturulmuştur. matplotlib kütüphanesi yardımıyla bu birleşmiş görüntü görselleştirilmiş ve hizalama işleminin başarısı gözlemlenmiştir. Bu adım, Prokudin-Gorskii yöntemiyle siyah-beyaz cam negatiflerden renkli görüntü elde etmenin temel sonucunu göstermektedir.

```
G_aligned, shift_G = align_channel(B, G, max_shift=15, metric='ssd')
R_aligned, shift_R = align_channel(B, R, max_shift=15, metric='ssd')

print("Green kaydırma:", shift_G)
print("Red kaydırma:", shift_R)

# Hizalanmış kanalları birleştir (RGB)
color_image = np.dstack([R_aligned, G_aligned, B])

# Sonuç göster
plt.figure(figsize=(6,4))
plt.imshow(np.clip(color_image,0,1))
plt.axis("off")
plt.title("Hizalanmış Renkli Görüntü")
plt.show()
```

Green kaydırma: (6, 2)
Red kaydırma: (6, 2)

Hizalanmış Renkli Görüntü



5. Görüntü İyileştirme ve Sonuç Aşaması

Bu bölümde, hizalanmış renkli görüntü üzerinde çeşitli görüntü iyileştirme işlemleri gerçekleştirilmiştir. İlk olarak, görüntünün kenarlarında oluşabilecek bozulmaları azaltmak amacıyla kenar kırpması (crop) işlemi uygulanmıştır. Ardından görüntü, normalize edilerek piksel değerleri 0–1 aralığına getirilmiştir.

Daha sonra, her bir renk kanalı için histogram eşitleme yapılmış ve böylece kontrast iyileştirilmiştir. Ek olarak, gama düzeltme (gamma correction) yöntemiyle görüntünün genel parlaklığı dengelenmiştir. Son adımda, hizalama sürecinden kaynaklanabilecek siyah boşlukları kaldırmak için otomatik siyah kenar kırpması işlemi uygulanmıştır.

Bu ardışık işlemler sonucunda, Prokudin-Gorskii yöntemiyle oluşturulan renkli fotoğrafın daha dengeli, kontrastlı ve doğal görünümü nihai hali elde edilmiştir.

```
def crop_edges(img, crop_ratio=0.10):
    """Kenarları kırpar"""
    h, w, _ = img.shape
    h_crop = int(h*crop_ratio)
    w_crop = int(w*crop_ratio)
    return img[h_crop:h-h_crop, w_crop:w-w_crop]

def normalize(img):
    """Piksel değerlerini 0-1 aralığına normalize eder"""
    return (img - np.min(img)) / (np.max(img) - np.min(img))

def hist_equalize(img):
    """Histogram eşitleme (her kanal ayrı)"""
    out = np.zeros_like(img)
    for i in range(3):
        out[:, :, i] = exposure.equalize_hist(img[:, :, i])
    return out

def gamma_correction(img, gamma=1.2):
    """Gama düzeltme ile parlaklığı ayarlar"""
    return img ** gamma

def auto_crop_black(img, threshold=0.10):
    """Siyah kenarları otomatik kırpar"""
    mask = np.max(img, axis=2) > threshold
    rows = np.any(mask, axis=1)
    cols = np.any(mask, axis=0)
    rmin, rmax = np.where(rows)[0][[0, -1]]
    cmin, cmax = np.where(cols)[0][[0, -1]]
    return img[rmin:rmax+1, cmin:cmax+1]

# Pipeline
color_cropped = crop_edges(color_image)
color_norm = normalize(color_cropped)
color_hist = hist_equalize(color_norm)
color_gamma = gamma_correction(color_hist, gamma=1.2)
final_img = auto_crop_black(color_gamma)

# Nihai sonuç gösterimi
plt.figure(figsize=(6,4))
plt.imshow(np.clip(final_img, 0, 1))
plt.axis("off")
plt.title("İyileştirilmiş Nihai Görüntü")
plt.show()
```

İyileştirilmiş Nihai Görüntü



6. Görüntülerin Karşılaştırmalı Görselleştirilmesi

Bu adımda, Prokudin-Gorskii renkli fotoğraf restorasyonu sürecinin farklı aşamaları görsel olarak karşılaştırılmıştır. matplotlib kütüphanesi kullanılarak üç farklı görüntü yan yana gösterilmiştir:

1 Orijinal Hizalanmamış Görüntü – Kanalların herhangi bir hizalama veya iyileştirme işlemi uygulanmadan önceki hali.

2.Hizalanmış Görüntü – R, G ve B kanallarının doğru biçimde üst üste getirilmesiyle elde edilen renkli görüntü.

3.İyileştirilmiş Nihai Görüntü – Kontrast, parlaklık ve kenar kırpmaya işlemleri sonrası elde edilen son hali.

Bu görselleştirme, uygulanan işlemlerin etkisini net bir biçimde ortaya koyarak proje sonucunun başarısını görsel olarak desteklemektedir.

```
fig, axes = plt.subplots(1,3, figsize=(15,10))
axes[0].imshow(np.clip(np.dstack([R,G,B]),0,1))
axes[0].set_title("Orijinal Hizalanmamış")
axes[1].imshow(np.clip(color_image,0,1))
axes[1].set_title("Hizalanmış")
axes[2].imshow(np.clip(final_img,0,1))
axes[2].set_title("İyileştirilmiş Final")
for ax in axes:
    ax.axis("off")
plt.show()
```

Orijinal Hizalanmamış



Hizalanmış



İyileştirilmiş Final

